

Project 3: A Particle Detector

Project Report

This report is coursework for PHYS30672 Object-Oriented Programming in C++

Abstract

A simulation of a particle detector was made and tested. The key components were two pure *abstract* base classes: `Particle` and `Subdetector`. Multiple classes were derived from each, which *inherited* and *overrode* the base class' *public* virtual member functions, and declared their own *private* members. A `Detector` class was also constructed, primarily as a container for `Subdetector` instances, which contained an array of *base pointers* to `Subdetector`-derived class instances. The simulation was reproducibly compiled, tested and found to work correctly. Multiple features beyond the assignment brief were also present.

Introduction

As part of the coursework for this module, a set of three projects was assigned to every student of which one was chosen. The objective given in this project was to create a simulation of a particle detector that met certain design criteria:

- The use of a class hierarchy with pure abstract base classes for subdetectors and particles.
- The derivation of each of these classes to specific types of the base class: i.e electrons, muons and hadrons for the particle class and muon chambers, calorimeters and trackers for the subdetector class. The functions in these derived classes override virtual functions defined in the respective abstract base class.
- The presence of a detector class that wraps multiple subdetector-derived class instances.
- The detection behaviour of particles corresponds to their real physical attributes.

Additionally, a functionality criterion was also required:

- The detector class is capable of wrapping any configuration of subdetector-derived class instances the user specifies.

The procedure for testing the program was also specified, which will be shown and followed later in the report.

The nature of particle accelerators, detectors, and their subdetector components was modelled in this experiment based on information about the CMS particle detector at CERN obtained from the CMS website, which was suggested by the project brief [1].

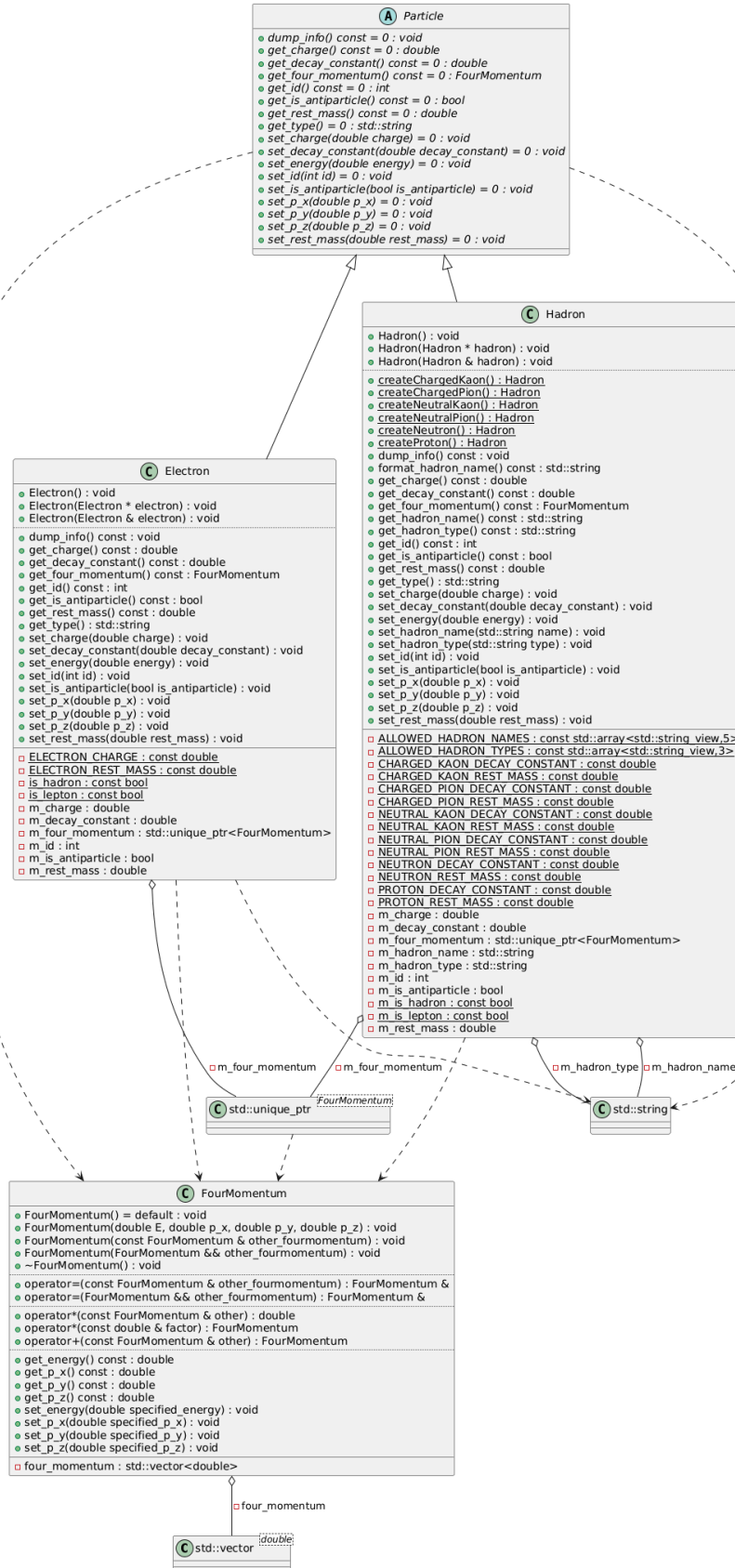
Design

Particle class hierarchy

This program made use of class hierarchies headed by pure *abstract* base classes. A class is pure abstract when it has no data members, exclusively declares virtual functions (with no implementation), and cannot be instantiated. The pure abstract class is utilised by creating derived classes that *inherit* and override the virtual functions, have their own data members, and can be instantiated. The key design advantage of this is the ease of modification of the program by a third party. The virtual functions in the abstract base class serve as a “minimum requirement” template for the functions an arbitrary derived class should have in order to interface with the rest of the program. A user should use this as a criteria for the completeness of a derived class they would wish to add. There is also a functionality advantage, as this kind of hierarchy allows intuitive and productive use of base pointers, which is demonstrated later in the program as an application of *polymorphism*.

A UML diagram of the hierarchy for the `Particle` and some derived particle types is given in Figure 1.

Figure 1: A UML inheritance diagram of the class hierarchy for particles, truncated to avoid excessive duplicate information (i.e leptons are demonstrated by just the Electron class). Private data members have a red marker.



Encapsulation is used in the design by defining all derived class data members as private and restricting external access to public setter and getter functions defined in the base class, allowing for managed access and modification of data members. This simplifies implementation of features such as input validation or output formatting. The `Hadron` class has additional features to those present in the `Electron` class. Named constructors are present in class `Hadron`, following the *named constructor idiom* [2]. A named constructor is a constructor called with a unique function name that instantiates a specific set of values for its data members. In the case of the `Hadron` class, they are used to instantiate common types of hadrons: protons, neutrons, and both charged and neutral variants of pions and kaons. This aids code readability and prevents code repetition. A code snippet of a `Hadron` named constructor is given in Snippet 1.

```
static Hadron createNeutron() // Named constructor to create a Neutron
{
    Hadron returned_hadron = Hadron();
    returned_hadron.set_id(2);
    returned_hadron.set_charge(0);
    returned_hadron.set_rest_mass(NEUTRON_REST_MASS);
    returned_hadron.set_decay_constant(NEUTRON_DECAY_CONSTANT);
    returned_hadron.set_hadron_type("baryon");
    returned_hadron.set_hadron_name("neutron");
    return returned_hadron;
}
```

Snippet 1: The named constructor that creates a `Hadron` instance whose data members match the parameters of a neutron.

A feature shown in Snippet 1 is the use of variable names to refer to constants. `NEUTRON_REST_MASS` and `NEUTRON_DECAY_CONSTANT`, used as arguments for their respective setters, are symbols that point to `static constexpr` data members in the `Hadron` class that contain the appropriate value. Contrarily, the constructor also contains magic numbers for the argument to the setters of the `m_id` and `m_charge` data members. This is a careful choice. `m_id` values are by nature magic numbers that refer to a certain type of particle, with the corresponding particle type being defined in the comments for the `Particle` base class. It was chosen to represent particle types this way (instead of using strings) to allow easier iteration through ID values and to save runtime memory. The disadvantage is that a lookup table of ID values to particle properties may be needed, and to preserve locality, these will not be in a global scope.

The physical constant data members mentioned earlier are used for good reason. The precision of particle charge is controlled entirely by the precision of the elementary charge unit (which is used by this program, hence charge values for all particles are integers). The value of the elementary charge unit is known to a very high degree of precision. However, for some particles, rest mass and decay constant values are relatively imprecise and accepted values may be subject to change as more precise measurements are made in the real world.

Hence, this program allows for changes in these values to easily take effect. For those values already known to high precision, it allows the user to easily increase the precision arbitrarily. Given the values are also data members, they can be used in member functions of the Hadron class as well should the user wish. This design element is also replicated in the lepton-type derived classes.

Another design element for particles is the asymmetry between setting and getting four-momenta components. Operations with four-momenta are likely to access all components of four-momentum, so the only appropriate getter returns (a deep copy of) the FourMomentum object itself, where FourMomentum is a class corresponding to four-momentum complete with its own setters and getters. Setting four-momenta is done by setting each component with the appropriate setter function. This asymmetry is intentional and exists to aid code readability, as it provides a briefer way for Particle-derived classes to copy the four-momentum of other instances.

Detector-subdetector class hierarchy

The UML inheritance diagram for the detector-subdetector class hierarchy is given in Figure 2.

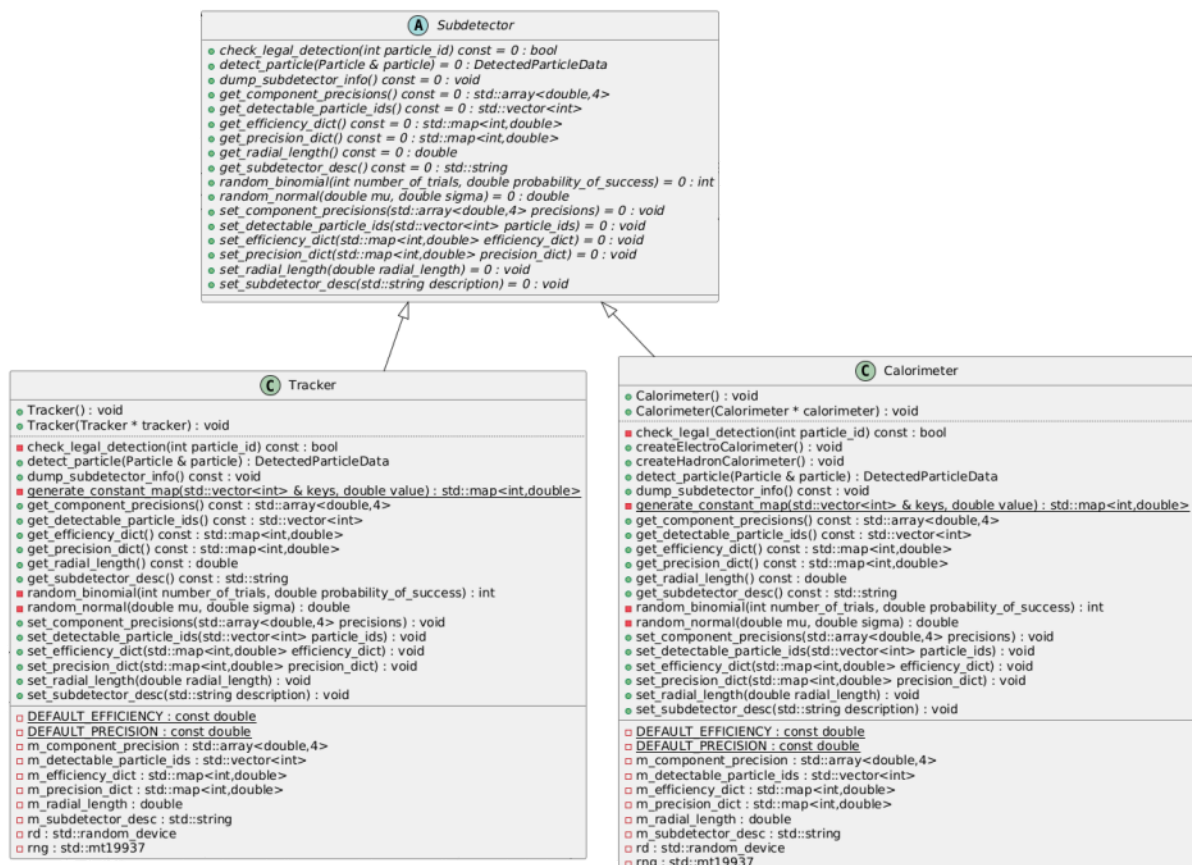


Figure 2: The truncated UML inheritance diagram for the Subdetector pure abstract base class. The MuonChamber class has been truncated due to its similarity to Tracker. Private member functions and private data members have filled and hollow red markers respectively.

The bulk of the code is located in the derived classes of the Subdetector pure abstract base class. The Detector class is essentially a container of Subdetector-derived class instances that carries some extra features which will be discussed later.

DetectedParticleData struct

This program uses a struct object named DetectedParticleData in order to simulate the detection data outputted by a subdetector upon detecting a particle. The definition of this data structure is given in Snippet 2.

```
struct DetectedParticleData
{
    DetectedParticleData operator+(const DetectedParticleData & x);
    void dump_detected_particle_info();

    bool from_calorimeter {false};
    FourMomentum detected_four_momentum {FourMomentum()};
    std::array<double, 4> detector_component_precision {1,1,1,1};
    std::vector<int> candidate_particle_ids {0};
    double travel_distance {0};
};
```

Snippet 2: The definition of the DetectedParticleData struct.

The structure incorporates subdetector imprecision as a data member, down to values for each component of four-momentum. The structure also has a definition for the addition operator, that aggregates and averages variable values over the two DetectedParticleData operands, including errors using the standard error relation. It also takes into account the inability of the Calorimeter type to detect momentum components (using the from_calorimeter variable, and will not propagate errors/values for momentum components for calorimeter detections. The default constructor of DetectedParticleData behaves as an additive identity, and is essentially a “null vector” of the structure, used when a subdetector fails to detect a particle. There is also a candidate_particle_ids vector, which contains all particle ID’s that are compatible with any of the detector types the particle has passed through, with duplicate ID’s in the vector intentionally allowed in order to retain information.

Subdetector hierarchy

Subdetectors are another example of a class hierarchy in this program headed by a pure abstract base class. The derived classes possess data members which control the important characteristics of the subdetector: Types of particle detected, detection efficiency, measurement precision (all three by particle ID), and precision multipliers for each four-momentum component. There are also members which add to the user interface, like the variable m_subdetector_desc and dump_subdetector_info(). The most important function to the objective of this project is detect_particle(). This is a function that simulates the detection of a particle that is passed as an argument to it. Its output is an instance of the DetectedParticleData structure which corresponds to the data the simulated subdetector

outputs. This function is overridden for derived classes in a way that replicates the behaviour for that type of subdetector. For example, `Calorimeter` instances will output only the energy component of four-momentum into its `DetectedParticleData` structure, while `Tracker` instances will output all components, but with greater imprecision by default on the energy component, as it does not detect it directly.

Subdetector efficiency and precision

Detection efficiency and measurement precision are modelled by this program.

When a subdetector receives a particle, a binomial distribution with $(N, p) = (1, E)$ where E is the detection efficiency is sampled from within the subdetector object to determine if the particle is detected. The function used to sample the distribution has a variable number of trials as it would be useful for a user wishing to extend the functionality of the program.

A measurement of a component of four-momentum by a subdetector is also subject to its imprecision. By default, this is 1% for each four-momentum component, except for the energy component for the `Tracker` type, where it is 3%, as trackers do not directly measure energy. A random offset is then added to each measured four momentum component by sampling a normal distribution with $(\mu, \sigma) = (0, \text{component precision} * \text{component value})$. The offset components are then returned.

Detector class

The key data member of the `Detector` class is `subdetector_array`, which contains an array of base pointers to instances of derived classes of `Subdetector` corresponding to different subdetector types. This array can be added to using the `append_subdetector()` or `prepend_subdetector()` functions, which make use of the *template* feature to automatically create overflows for all derived classes of `Subdetector` at compile time. This is more compact and readable than manually creating overflows, since both functions only create a derived class object of `Subdetector` managed by a `unique_ptr` that is moved into the `subdetector_array`. The key member function of `Detector` is `transmit_particle()`, which iterates through this array of base pointers and calls the `detect_particle()` function for each value of the iterator. It sums the returned `DetectedParticleData` structs into one single detection that is then returned by the function.

Detector: `resolve_particle()`

An extra feature this program has is the ability of `Detector` objects to attempt to resolve the type of particle by taking the propagated `DetectedParticleData` and testing it against numerous criteria. It begins by first checking if the particle was detected by a tracker. Trackers only detect charged particles, therefore any candidate particles that are neutral are discarded from the possibilities. From the remaining candidates, the one with the closest (invariant) mass to the detection is selected on the conditions it lies within a given number (5 by default) of standard deviations of the mass. Should this condition not be met, there are two fallbacks. If the particle could be an exotic hadron (whose masses vary depending on type),

that is given as the result. However, if this is not the case, and the particle is not detected by any tracker and is therefore likely neutral, the particle is resolved as a photon. Failing all these, the routine exits cleanly and prints an error message to the screen. The function also prints an estimate of the probability of the resolved particle having decayed during transmission, provided it is an unstable particle.

main()

```
int main()
{
    // (... definitions and declarations)
    for(auto it = test_hadrons.begin(); it != test_hadrons.end(); ++it)
    {
        particle_zoo.emplace_back(std::make_unique<Hadron>(&(*it)));
    }
    particle_zoo.emplace_back(std::make_unique<Electron>(test_electron));
    particle_zoo.emplace_back(std::make_unique<Muon>(test_muon));
    particle_zoo.emplace_back(std::make_unique<Photon>(test_photon));

    detector.dump_subdetector_config();

    for(auto it = particle_zoo.begin(); it != particle_zoo.end(); ++it)
    {
        std::cout<<"--- BEGIN NEW PARTICLE ---"<<std::endl;
        give_random_four_momentum(**it);
        DetectedParticleData current_detection = detector.transmit_particle(**it);
        current_detection.dump_detected_particle_info();
        detector.resolve_particle(current_detection);
        (**it).dump_info();
    }
    return 0;
}
```

Snippet 3: The (truncated) script used to test the program. The `particle_zoo` variable is an array of base pointers to all possible types of particles well-defined by the program, including all leptons (specified by their derived class) and hadrons (by named constructors). Each particle has been given a random four-momentum. It is iterated over, and each dereferenced instance is simulated being transmitted through the detector. The averaged sum of all detections is printed using the appropriate function in `DetectedParticleData`. The `resolve_particle()` function is applied to the summed-up `DetectedParticleData` output, which prints the name of the closest matched particle to the detection, if found. The true particle info is then printed to the screen. A marker is then placed to inform the user that a new particle is being tested.

The testing script relies entirely on the behaviour of base pointers to derived class instances (in `detector.subdetector_array`). The functions called on the derived classes to test the program as the iteration takes place are (by design) all overrides from the abstract base class.

When functions are called on the dereferenced base pointer, they automatically jump to the appropriate derived class' override of the base class function. This feature would not be present if the base class was not abstract. This feature also enables functions to take a base class instance as an argument that generalises to all derived classes. Both Subdetector and Particle hierarchies utilise this.

Results

The program compiled under g++ with the -Wall flag successfully and without throwing warnings. The program executed without error and exited cleanly. The output of the program is given in Snippet 4.

```
-- Detector configuration --
- Radially outward: -
- (inner detector) -
- tracker -
- hadron calorimeter -
- electromagnetic calorimeter -
- muon chamber -
-----
--- BEGIN NEW PARTICLE ---
-- Detection data --
Entirely calorimeter detection: 1
Detected (E, p_x, p_y, p_z): (3894.67, 0, 0, 0)
Precision: (0.01, 0, 0, 0)
Candidate particle ID's: 1, 2, 3, 4, 5, 6, 12,
-----
-----Detector particle resolution routine-----
Resolved particle (by negative elimination) : Exotic Hadron
-----
--- Hadron info ---
Type: baryon
Formatted name: proton
Charge: 1
Is antiparticle: 0
Four momentum: (E, p_x, p_y, p_z) = (3916.25, 732.45, 2730.76, -2542.37)
Decay constant: 0
ID: 1
-----
--- BEGIN NEW PARTICLE ---
-- Detection data --
Entirely calorimeter detection: 1
Detected (E, p_x, p_y, p_z): (2619.01, 0, 0, 0)
Precision: (0.01, 0, 0, 0)
Candidate particle ID's: 1, 2, 3, 4, 5, 6, 12,
-----
-----Detector particle resolution routine-----
Resolved particle (by negative elimination) : Exotic Hadron
-----
--- Hadron info ---
Type: baryon
Formatted name: neutron
Charge: 0
Is antiparticle: 0
Four momentum: (E, p_x, p_y, p_z) = (2611.06, 715.985, -2323.96, 143.539)
Decay constant: 0.00113
```

Snippet 4: A small segment of the output of `main()`, dumping the `DetectedParticleData` corresponding to each particle transmitted through the detector, followed by the particle type resolved by detector, and the true information about the detected particle. The script that leads to this output is found in Snippet 3.

The `resolve_particle()` function is successful in acquiring the correct particle type for most charged particles, but fails every time when attempting to resolve neutral particles that are not photons. This is because, since neutral particles are only detected by `Calorimeter`, which only measures energy, their invariant masses are unobtainable. As may possibly be seen when running the program, it is possible for charged pions and muons to be interchanged by `resolve_particle()`, this is due to their close proximity in mass (~ 30 MeV) combined with the detector imprecision causing a false positive in the mass matching routine.

Results do vary between each run due to the imperfect detection efficiency of subdetector components, which allows for the possibility of particles to be undetected regardless of particle type. Where this occurs, the test routine handles it gracefully. The routine executes very fast (sub-second), so re-running it is easy if particles are undetected due to detector inefficiency.

Evaluation and conclusion

The program was tested to success within its scope. A criticism of the code is overcommitment to locality. This is evident in the `Detector` class, where lookup tables are generated of particle attributes in-class as private data members. These are unable to be made `static` or `static constexpr`, due to being of `std::map` type and are generated for each instance. In this case, it may be sensible to use `enum` in a higher (not global) scope so that these lookup tables can change from one origin and propagate throughout the program. The use of external (`.ini`, `.dat`, `.cfg` etc.) files is another possible solution.

A possible extension to the program could be a structure to represent final states of particle physics events. This would ideally be a `struct` object containing a container of derived classes of `Particle` with physically meaningful attributes, and functions to take the total charge, four-momentum etc. of the event would be sensible. This could then be used to simulate the behaviour of real particle physics events in the detector.

In conclusion, the program was successful within its appropriate scope, with some extra features adding some physical usefulness too. There is much space for extension to the program, which is expedited by its modular design and class hierarchy system, though it does suffer from underutilisation of language features (`enum`).

References

[1]: CMS Collaboration, “CMS Detector”, cms.cern, <https://cms.cern/index.php/detector> (accessed 13th May 2026)

[2]: Standard C++ Foundation, C++ Super-FAQ, <https://isocpp.org>, <https://isocpp.org/wiki/faq/ctors#named-ctor-idiom> (accessed 15th May 2026)

The word count in this report excluding references, captions, code snippets and this sentence is 2473 words.